Lab 1 Tasks: The Morse Code Lock

GPIO and the Memory Map

Contents

Overview 1

Objectives 2

Required Hardware 2

Pre-Laboratory Exercises 2
- Part "A": Inline Assembly . . . 3
- Part "B": Register Access in the C Language . . . 3
- Part "C": The HAL . . . 3

Part "A": Inline Assembly 4
- **Task A1: Blink LD2 with assembly code** . . . 4
- **Task A2: Move the count into SRAM and measure the cost** . . . 5

Part "B": Register Access in the C Language 6
- **Task B1: Control the blink speed with the user button** . . . 6
- **Task B2: Use D2 as the button** . . . 7
- **Task B3: Debounce D2** . . . 7

Part "C": The HAL 9
- **Task C1: Enter your name in Morse code** . . . 9

Demonstration and Sign-off 10

Overview

The development board becomes a combination lock. The key is your first name in Morse code. You enter your name on the user button B1 as a sequence of short and long presses. The program decodes the presses and compares the result with the stored name. The user LED LD2 shows the result (Figure 1):

- If the name is correct, LD2 blinks quickly. The lock is open.
- If the name is not correct, LD2 stays on. The lock stays closed.

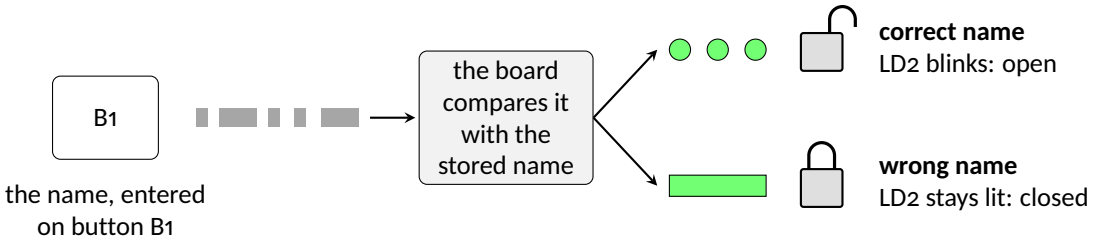


Figure 1: The Morse code lock. The grey bars show short and long presses of button B1. The program compares the entered name with the stored name and shows the result on LD2.

Structure of this laboratory

This laboratory has three parts.

- **Part "A", inline assembly.** Make LD2 blink with assembly code. With the oscilloscope, measure the time that LD2 stays on.
- **Part "B", register access in the C language.** Control the blink rate with a push-button. Then measure the contact bounce of the input, and learn how to remove it.
- **Part "C", the hardware abstraction layer (HAL).** Build the lock with the HAL functions from STMicroelectronics (ST).

Objectives

- To show that memory-mapped registers control a general-purpose I/O (GPIO) pin, and that load and store instructions access these registers.
- To find the address, the bit fields, and the reset value of a register in the reference manual.
- To configure and drive a GPIO pin with inline assembly.
- To predict a software delay from instruction cycle counts, and to measure the delay with an oscilloscope.
- To measure how much time a value in SRAM costs, compared with a value in a core register.
- To read a push-button through its registers, and to debounce it.
- To measure the duration of button presses with the HAL, and to decode Morse code.

Required Hardware

- The NUCLEO-G491RE development board.
- An oscilloscope.
- Four male-to-male jumper wires: one for ground, one for LD2, and two for the button.

Pre-Laboratory Exercises

Do the pre-laboratory exercises before the laboratory session. Each part has its own exercises. Read the part in the booklet first.

Part "A": Inline Assembly

1. **Predict the blink.** Tasks A1 and A2 measure the *half-period* of the blink. The half-period is the time that LD2 stays on. The two tasks do the same delay loop $N = 56\,666\,667$ times on a 170 MHz core. Only the instructions in one pass are different:

Task	One pass of the loop
A1, count in a register	`subs`, `bne`
A2, count in SRAM	`ldr`, `subs`, `str`, `bne`

   The half-period is $N \times c/f$. Here, c is the number of clock cycles of one pass, and f is the clock frequency.

   (a) Find the cycles of each instruction in Table 3-1 of the Arm Cortex-M4 Technical Reference Manual (DDI 0439 section 3.3.1, p. 29). For `str`, use the section *Load/store timings* (DDI 0439 section 3.3.2, p. 36). For `bne`, use the smallest value for a taken branch.
   (b) Calculate c and the half-period for each task.
   (c) Which task blinks more slowly? Explain why in one sentence.

Part "B": Register Access in the C Language

2. **Count the presses.** A program reads button B1 on each pass of its loop. It reads bit 13 of `GPIOC_IDR`: `1` while the button is held, and `0` when it is released. Ten passes read `0 0 1 1 1 0 0 1 1 0`.

   (a) Write a C expression that gives bit 13 of `GPIOC_IDR` as `0` or `1`.
   (b) Give the number of presses in the ten passes. A press is a rising edge: a `0`, then a `1`.
   (c) Write the condition that is true only on a pass with a rising edge. Use `now`, the value of this pass, and `previous`, the value of the pass before it.
   (d) At one press, the contact bounces, and six passes read `0 1 0 1 1 1`. Give the number of presses that your condition counts.

Part "C": The HAL

3. **Encode your name.** Use the Morse code table (Table 6 of the booklet). Write your first name in Morse code as the string that your program compares in Task C1. Write the dots and dashes of each letter, with one space between letters. Do not put a space at the start or at the end. For example, YUVAL is `"-.-- ..- ...- .- .-.."`.

4. **Find the limit of the counter.** `HAL_GetTick()` returns a `uint32_t` counter that increments each millisecond.

   (a) Calculate the time, in days, after which the counter goes back to 0.
   (b) A press starts when the counter is `0xFFFFFFF0`, and it ends when the counter is `0x00000010`. Calculate `end - start` as a `uint32_t`. Is this the duration of the press?

Part “A”: Inline Assembly

Task A1: Blink LD2 with assembly code

Run the code below without changes. It makes LD2 blink. Measure the duration of each half-period with the oscilloscope.

```
__asm volatile (
    /* RCC_AHB2ENR: set GPIOAEN (bit 0) */
    "ldr   r6, =0x4002104C   \n"
    "ldr   r5, [r6]          \n"
    "orr   r5, r5, #0x1      \n"
    "str   r5, [r6]          \n"

    /* GPIOA_MODER: MODE5 (bits 11:10) = 01, clear then set */
    "ldr   r0, =0x48000000   \n"   /* r0 = GPIOA base */
    "ldr   r5, [r0]          \n"
    "bic   r5, r5, #0xC00    \n"
    "orr   r5, r5, #0x400    \n"
    "str   r5, [r0]          \n"

    /* GPIOA_ODR (base + 0x14): set bit 5, LD2 on */
    "ldr   r5, [r0, #0x14]   \n"
    "orr   r5, r5, #0x20     \n"
    "str   r5, [r0, #0x14]   \n"

    /* One half-period, then flip bit 5 and do it again */
    "1:                      \n"
    "ldr   r3, =56666667     \n"   /* passes per half-period */
    "2:                      \n"
    "subs  r3, r3, #1        \n"
    "bne   2b                \n"
    "ldr   r5, [r0, #0x14]   \n"
    "eor   r5, r5, #0x20     \n"
    "str   r5, [r0, #0x14]   \n"
    "b     1b                \n"
    ".ltorg                  \n"   /* literal pool, never executed */
    :
    :
    : "r0", "r3", "r5", "r6", "cc", "memory"
);
```

Listing 1: Task A1: The code enables the GPIOA clock, configures PA5 as an output, and makes LD2 blink, in inline assembly. Put the block in the while (1) loop of main().

Structure of the code. Blank lines divide the code into four groups. The groups do these operations:

- enable the GPIOA clock in `RCC_AHB2ENR`.
- configure PA5 as an output in `GPIOA_MODER`.
- turn LD2 on in `GPIOA_ODR`.
- blink: count `r3` down from 56 666 667, toggle PA5, and do it again.

Each of the first three groups is a read-modify-write sequence.

Steps.

1. Put the code in `main.c`, between `USER CODE BEGIN WHILE` and `USER CODE END WHILE`. Build the project. Flash the board.
2. Make sure that LD2 comes on and then blinks.
3. Probe PA5. Measure the half-period. Compare it with your prediction from Pre-Laboratory Exercise 1(b).
4. Use Table 3 of the booklet to find the effect of each instruction in the code.

Task A2: Move the count into SRAM and measure the cost

Store the count in memory, and measure the increase of the half-period. The count stays 56 666 667. Thus, the only difference from Task A1 is where the count is: in SRAM, and not in a core register.

```
uint32_t blink_count;   /* USER CODE BEGIN PV; lives in SRAM */

        "ldr   r4, =blink_count  \n"   /* r4 = its address */
```

Listing 2: Task A2: The global variable for the count, in SRAM, and the instruction that loads its address.

Steps.

1. Declare `blink_count` in the `USER CODE BEGIN PV` section, as the code above shows.
2. Before the blink loop, load the address of `blink_count` into `r4` one time. Add `r4` to the clobber list.
3. Change the delay loop so that `blink_count` holds the count. At the start of each half-period, store the count in `blink_count`. On each pass, load the count, subtract 1, and store it again. **Do not change the other code.**
4. Build the project. Flash the board. Measure the half-period with the oscilloscope, as in Task A1. Calculate its ratio to the half-period of Task A1.
5. In the debugger, find the address of `blink_count`. Find the region of the memory map that contains this address.
6. For each of the two loops, calculate the number of cycles per pass. Use the count, the clock frequency of 170 MHz, and the half-period that you measured.
7. Compare the two measurements with your predictions from Pre-Laboratory Exercise 1. Explain the difference between the two loops. Use the instructions in each pass, and where each instruction gets its data: a core register or memory.

Part "B": Register Access in the C Language

In Part "B", you write C code that accesses each register through its address . Do not use `HAL_*` functions. The program operates as follows:

- At reset, LD2 is off.
- The first press starts a slow blink.
- Each press after the first press divides the half-period by 2. There are five speeds. At the fastest speed, the next press sets the blink to the slowest speed again.
- The program counts loop passes to time the blink, as in Part "A". The loop does not wait: it reads the input on each pass.

Task B1 uses the user button as the input. Tasks B2 and B3 use two jumper wires. One of the wires is on PA10.

Task B1: Control the blink speed with the user button

Write the program that Part "B" describes. Use a rising edge to find each press.

```
/* USER CODE BEGIN PD */
#define RCC_AHB2ENR   (*(volatile uint32_t *)0x4002104CUL)
#define GPIOA_MODER   (*(volatile uint32_t *)0x48000000UL)
#define GPIOA_BSRR    (*(volatile uint32_t *)0x48000018UL)
#define GPIOC_MODER   (*(volatile uint32_t *)0x48000800UL)
#define GPIOC_IDR     (*(volatile uint32_t *)0x48000810UL)

#define SLOW_HALF_PERIOD   3000000UL   /* passes, slowest speed */
#define FAST_HALF_PERIOD    187500UL   /* passes, fastest speed */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint32_t half_period = 0;    /* passes per half-period; 0 = not started */
uint32_t countdown   = 0;    /* passes left in this half-period */
uint32_t led_on      = 0;    /* 1 while LD2 is lit */
uint32_t previous    = 0;    /* B1's level on the previous pass */
/* USER CODE END PV */
```

Listing 3: Task B1: Register macros, speed limits, and variables. Put the define lines in USER CODE BEGIN PD, and the variables in USER CODE BEGIN PV.

Steps.

1. Copy the code above into the two sections that its comments give.
2. In `USER CODE BEGIN 2`, which runs one time, enable the GPIOA and GPIOC clocks. Configure PA5 as an output and PC13 as an input. Clear each field before you set it. Turn LD2 off through `GPIOA_BSRR`.
3. In the `while (1)` loop, write one pass of the loop. The pass:
   - reads button B1 from `GPIOC_IDR`.
   - on a rising edge, starts the blink or changes to the next speed.
   - stores this value in `previous`.
   - after the blink starts, decrements `countdown`. When `countdown` is zero, the pass toggles LD2 through `GPIOA_BSRR` and loads `countdown` from `half_period` again.

Task B2: Use D2 as the button

Two jumper wires replace the button. Put one wire into D2 (PA10) and the other wire into a 3V3 socket. When the free ends of the wires make contact, the circuit closes: this is a press. When the ends separate, this is a release. **Do not connect a wire to a 5V socket.** The code below configures PA10 as the input. The other code stays as in Task B1.

```
/* USER CODE BEGIN PD: add these two macros */
#define GPIOA_PUPDR   (*(volatile uint32_t *)0x4800000CUL)
#define GPIOA_IDR     (*(volatile uint32_t *)0x48000010UL)

  /* USER CODE BEGIN 2: add after the Task B1 setup */
  GPIOA_MODER &= ~(0x3UL << (10 * 2));      /* MODE10 = 00: input */
  GPIOA_PUPDR &= ~(0x3UL << (10 * 2));      /* PUPD10: clear */
  GPIOA_PUPDR |=  (0x2UL << (10 * 2));      /* PUPD10 = 10: pull-down */

    /* while (1): replace the line that reads B1 */
    uint32_t now = (GPIOA_IDR >> 10) & 1UL;   /* D2: 1 = in contact */
```

Listing 4: Task B2: The three changes that configure PA10 as the input. Each comment gives the location of the change in main.c.

Steps.

1. Make the three changes. Build the project. Flash the board.
2. Put one wire into D2 and the other wire into 3V3. Hold one wire in each hand, and bring the two free metal ends into contact several times. LD2 starts to blink, and each contact changes the blink rate.
3. Probe PA10. Capture the signal when the wires make contact and when they separate. Measure the settling time of each.

Task B3: Debounce D2

Make sure that the program counts each contact exactly one time. To do this, debounce D2. Accept a new level of PA10 only after it stays the same for longer than the bounce from Task B2.

```
/* USER CODE BEGIN PD */
#define DEBOUNCE_PASSES   /* your value */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
uint32_t accepted = 0;       /* debounced level of PA10 */
uint32_t disagree = 0;       /* passes PA10 has differed from it */
/* USER CODE END PV */
```

Listing 5: Task B3: The debounce threshold, which you calculate in step 3, and its two variables.

Steps.

1. Make contact between the wires ten times. Record the number of contacts that change the speed by more than one step.
2. Find the duration of one pass of your loop. At the slowest speed, measure the half-period of LD2 on PA5 with the oscilloscope. Divide it by `SLOW_HALF_PERIOD`.
3. Calculate `DEBOUNCE_PASSES` from this duration and the longer of the two settling times from Task B2. Include a safety margin. Record the calculation.
4. Replace the edge detection in your loop. Count each pass on which PA10 is different from `accepted`. Set the count to zero on each pass on which the two are the same. When the count is equal to `DEBOUNCE_PASSES`, copy the level of PA10 into `accepted`. Change the speed only if `accepted` is then `1`.
5. Build the project. Flash the board. Make contact between the wires ten times again. Include short contacts and contacts that slide. Each contact must change the speed by exactly one step.

Part "C": The HAL

Task C1: Enter your name in Morse code

Your program operates as follows:

- LD2 is on while button B1 is held.
- If a press is shorter than `DOT_MAX_MS`, it is a dot. If it is longer, it is a dash.
- If a pause is longer than `LETTER_GAP_MS`, it ends a letter. If it is longer than `NAME_GAP_MS`, it ends the name.
- The program then compares the entered name with your name. If they are the same, LD2 blinks quickly for `VERDICT_MS`. If they are different, LD2 stays on for `VERDICT_MS`.

```
/* USER CODE BEGIN Includes */
#include <string.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PD */
#define DOT_MAX_MS      300U   /* shorter press: dot; longer: dash */
#define LETTER_GAP_MS   800U   /* a longer pause ends a letter */
#define NAME_GAP_MS    2500U   /* a longer pause ends the name */
#define VERDICT_MS     2000U   /* how long LD2 shows the verdict */
/* USER CODE END PD */

/* USER CODE BEGIN PV */
static const char name_morse[] = ". ...- .-";  /* EVA: use your name */
static char keyed[64];           /* dots, dashes, letter spaces */
/* USER CODE END PV */
```

Listing 6: Task C1: The include directive, the time limits, your name, and the buffer for the input. Put each part in the section that its comment gives.

Steps.

1. In `USER CODE BEGIN MX_GPIO_Init_2`, enable the clocks of ports *A* and *C* with `__HAL_RCC_GPIOA_CLK_ENABLE()` and `__HAL_RCC_GPIOC_CLK_ENABLE()`. Then use `HAL_GPIO_Init()` to configure PA5 as a push-pull output and PC13 as an input. Do not use an internal pull resistor on PC13. Button B1 has an external pull-down resistor on the board.
2. Copy the code above into the sections that its comments give. Replace the example in `name_morse` with the Morse code of your name from Pre-Laboratory Exercise 3.
3. In the `while (1)` loop, write one pass of the loop. Use `HAL_GetTick()` as the only time reference. The pass:
   - reads button B1 with `HAL_GPIO_ReadPin()`, and turns LD2 on while button B1 is held.
   - records the time of each press. At each release, the pass adds a dot or a dash to `keyed`, as the duration of the press sets.
   - during a pause, adds a space to `keyed` after `LETTER_GAP_MS`. After `NAME_GAP_MS`, it removes the last space from `keyed`. Then it compares `keyed` with `name_morse`, shows the result for `VERDICT_MS`, and clears `keyed`.
4. Build the project. Flash the board. Enter your name. Then enter it with one incorrect letter. Make sure that the board rejects it.
5. Enter one letter. Pause the program in the debugger. Compare the contents of `keyed` with that letter in Table 6 of the booklet.

Demonstration and Sign-off

Keep these items, and be ready to show them to the instructor at the test of this lab:

- Your answers to Pre-Laboratory Exercises 1 to 4.
- For each task, the items of Table 1. An image of the oscilloscope screen can be a photo or a screenshot.

Task	Copy of `main.c`	Oscilloscope image
A1	`lab1partAtask1.c`	PA5: one half-period of LD2.
A2	`lab1partAtask2.c`	PA5: one half-period, with the count in SRAM.
B1	`lab1partBtask1.c`	No image.
B2	`lab1partBtask2.c`	PA10: two images, the bounce when the wires make contact (a press), and the bounce when they separate (a release).
B3	`lab1partBtask3.c`	PA5: one half-period at the slowest speed.
C1	`lab1partCtask1.c`	No image.

Table 1: The items to keep for each task.